

Web Systems Development

Week 4 · CSS: the cascade, the tools, the frameworks



September 15, 2026

AGENDA

Five parts

1

Business relevance of CSS

Conversion, search, accessibility and the law. Why anyone pays for a stylesheet.

2

DevTools for CSS

Six moves: inspect, read the cascade, edit live, force states, go mobile, keep what you found.

3

CSS frameworks

How a rule gets chosen; what Grant, Gallagher and Wathan say to do about it; then one page, five ways.

4

AI and design work

A loop that works, a prompt that works, what to reject, and how to give the assistant current knowledge.

Hand-in: inclass/wk4/ANSWERS.md, two questions, pushed before you leave. Friday: HW2 due, HW3 assigned.

Readings



Keith Grant, *CSS in Depth*, 2nd ed., ch. 9

Modular CSS and scope. Why the cascade makes every rule global, then the lineage of answers: modules, BEM, utilities, @scope. Ch. 1 if you cannot say why .nav a beats a.link. [RPI library](#), [O'Reilly](#).

Adam Wathan, CSS Utility Classes and “Separation of Concerns” (2017)

[The essay](#) that became Tailwind. He starts as a believer in semantic names and refactors his way off it. Read it to argue with it.

[Optional: Gallagher, About HTML Semantics and Front-End Architecture \(2012\)](#)

The position Wathan is arguing against, from the person who made it best.

The question the readings disagree on: should a class name say what a thing is, or what it looks like?

“It’s not a programming language in the conventional sense, but it does require abstract thought. It’s not purely a design tool, but it does require some creativity. It provides a deceptively simple declarative syntax, but if you’ve worked with it on any large projects, you know it can grow into unwieldy complexity.”

Keith Grant, CSS in Depth, 2nd edition





Thomas Fuchs 
@thomasfuchs

 **Follow**



Two CSS properties walk into a bar.

A barstool in a completely different bar falls over.

RETWEETS

2,806

LIKES

2,346



9:10 AM - 28 Jul 2014



48



2.8K



2.3K

One rule in checkout.css knocks over faq.html

checkout.css — the bar you walked into

```
/* make the Pay button pop */  
button {  
  width: 100%;  
  font-size: 24px;  
}
```



faq.html — a completely different bar

Was this helpful?

Yes

No

Yesterday: two small side-by-side buttons. Today: full width, 24px text. The barstool fell over.

CSS has no scope: the selector **button** matches every button on the site, so a fix in one bar tips a barstool in another.

1

Why should business care about CSS?

Conversion, search, accessibility. Three business problems that become pixels in the stylesheet.

CSS Matters for Marketing and Compliance

Conversion

How fast a page feels is decided in the stylesheet: layout shift, render-blocking CSS and font swaps all sit between a visitor and the buy button.

Brand is consistency at scale: one blue, one spacing scale, every page. A framework is how a team ships that.

Search

Google ranks pages on experience: how quickly they paint, how stable the layout is, how fast they respond. Most of that experience is set in CSS.

Google indexes the mobile page. The phone layout your CSS produces is the page Google reads.

Accessibility and the law

Low-contrast text is the most common accessibility failure on the web. Contrast, focus rings, reflow and target size are all CSS decisions.

Accessible styling is a legal obligation, not a courtesy: rules in the US and EU now apply to the web, and lawsuits follow the failures.

Not “make it pretty.” Revenue, reach and liability, decided by the stylesheet.

CSS choices directly affect revenue, reach and liability

CONVERSION

+8%

retail conversions from 0.1s
faster on mobile

Google / Deloitte, 2020

SEARCH

2 of 3

Core Web Vitals that are
mostly CSS

Google page experience

CONTRAST

4 in 5

home pages fail on text
contrast

WebAIM Million

LIABILITY

4,000+

US web-accessibility suits a
year

UsableNet

WHAT THE STYLESHEET CONTROLS

SPEED

aspect-ratio · font-display
transform, not top/left
ship less CSS

SEARCH

mobile-first layout that shows
the content instead of hiding it

CONTRAST

readable text contrast (4.5:1)
a visible keyboard-focus outline
layout that fits a phone (320px)
tap targets 24px or larger
less motion when users ask for it

LIABILITY

the same list as a checklist
your Playwright tests run

2

DevTools for CSS



MOVE 1

Open it, then point at the thing

1

Right-click, Inspect

DevTools opens with that element selected and its CSS in the Styles pane beside it.

right-click

Inspect

2

Or use the picker



The arrow-in-a-box icon at the top left. Hover shows the box and its size; click selects.

⌘⇧C

Ctrl+Shift+C

3

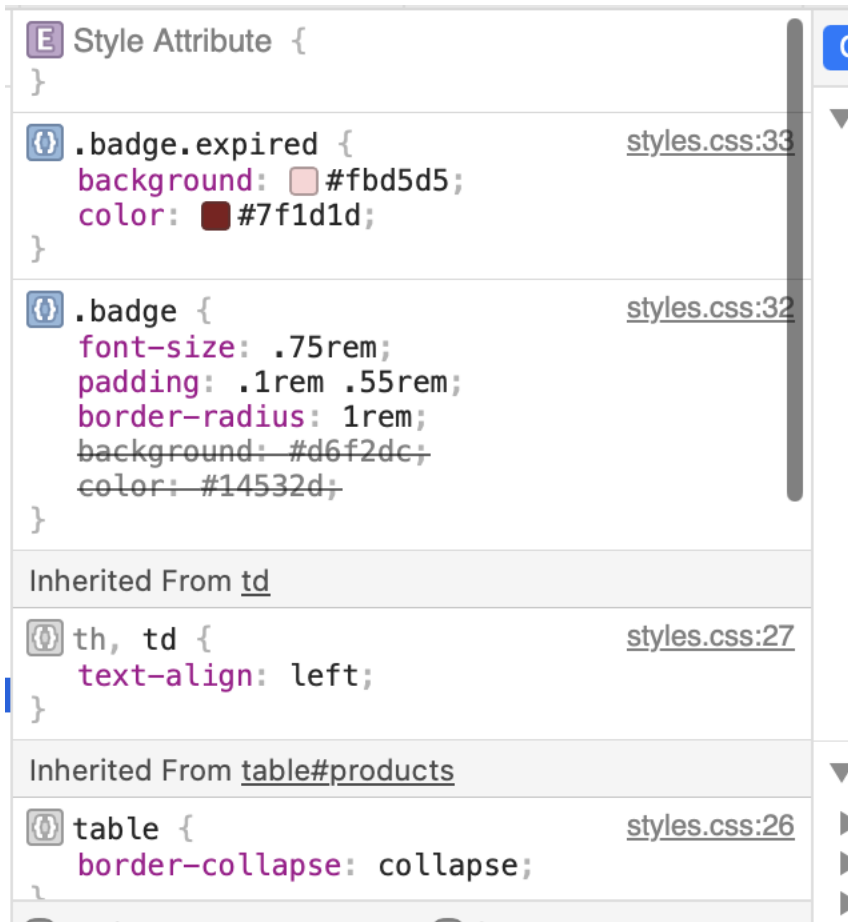
Mobile View



MOVE 2

Read the Styles pane: it is the cascade, drawn

Click on the “expired” in our bread example.



- Rules are listed in winning order: `.badge.expired` sits above `.badge` because it is more specific, so its red background is the one you see on the badge.
- The struck-through background and color in `.badge` lost: they matched, but the rule above overrode them.
- The grey rules under “Inherited from td” and “Inherited from table#products” were never written for the badge: `text-align` and `border-collapse` arrive from its ancestors.

MOVE 3

Edit on the fly: sketch before you touch the file

Click a value, type

Enter commits, Esc cancels, Tab moves on. The page updates as you type.

Arrow keys nudge numbers

Up and down by 1. Shift for 10, Alt for 0.1. Hold it and watch the layout move.

Untick a line to turn it off

The fastest answer to “what is this rule even doing?”

Click the swatch

Colour picker, eyedropper, contrast ratio against the background.

Add a line, add a rule

Click the blank space in a rule to add a line; the + adds a rule for this element.

Click file:line

Opens the stylesheet in Sources. Edits there apply live too.

Nothing here is saved. Reload and it is gone. It is a sketchpad for finding the fix, not the fix.

MOVE 4

Force a state, toggle a class, trace a number

:hov -- force a state

```
[ ] :active    [x] :hover  
[ ] :focus    [ ] :visited
```

Stays hovered while your mouse is in DevTools, so you can edit the hover rule.

.cls -- toggle classes

```
[x] badge  
[x] expired  
[ ] add new class...
```

Untick "expired" and watch the Styles pane reorder.

COMPUTED -- the final answer

```
background-color  
  rgb(229,231,235)  
> .shopping-list .badge
```

Every property, one value, and the rule that set it.

The box-model diagram at the bottom of Styles

Margin, border, padding, content, drawn with the real numbers. Double-click a number to edit it. Hover a band and the page highlights it: the quickest way to tell margin on this element from padding on its parent.

Shrink the page without shrinking the window

1

Toggle the device toolbar

The phone-and-tablet icon next to the picker. The page rerenders in a resizable frame.

⌘⇧M

Ctrl+Shift+M

2

Pick a width, or drag

Type 375 × 812, choose a real device, or drag the edge and watch the breakpoint flip.

375

768

1280

3

Show media queries

Three-dot menu in the toolbar. One coloured bar per breakpoint in your CSS; click to jump.

4

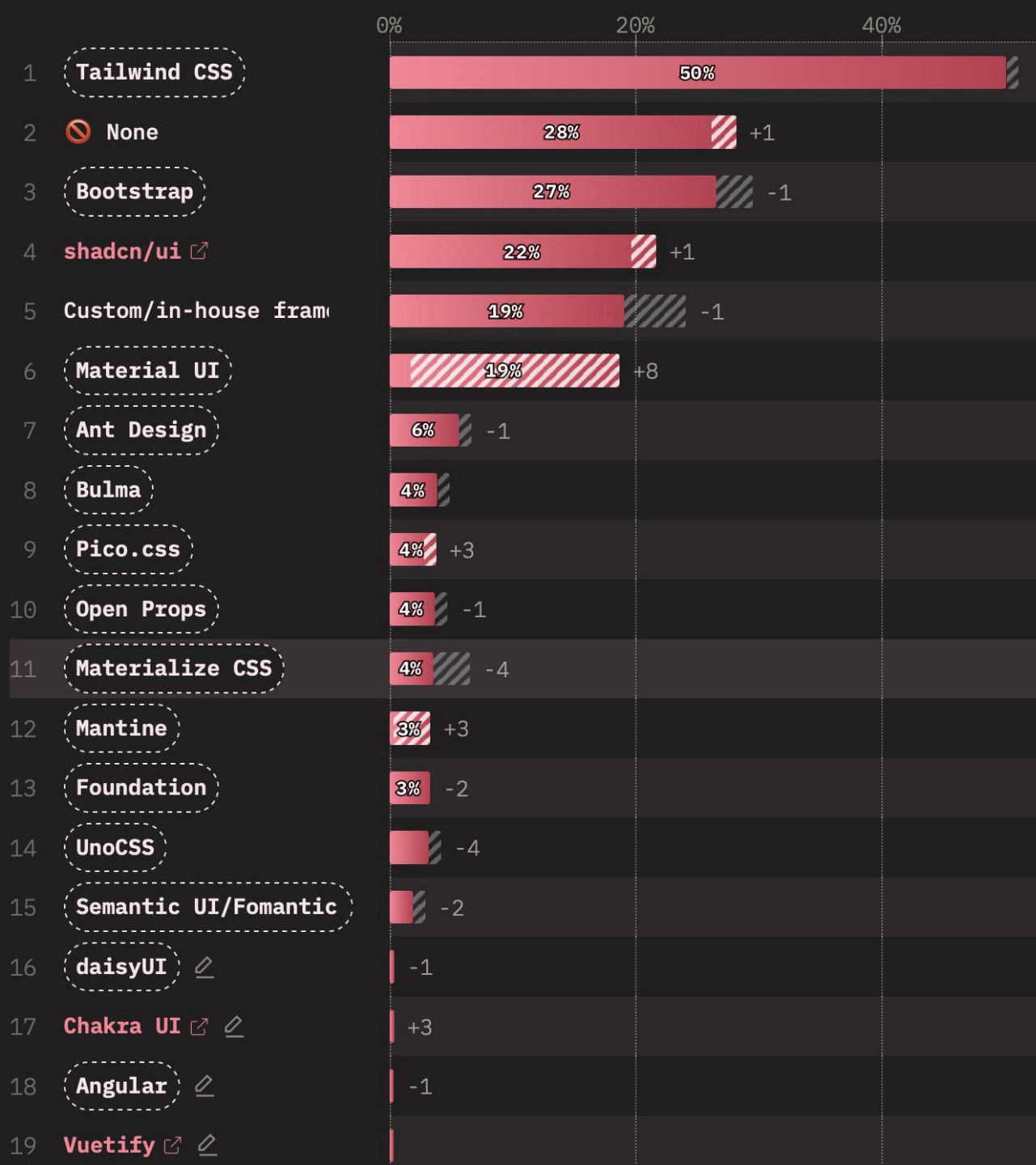
Capture screenshot

Same menu. Full-size screenshot at that width: what the shots service does with Playwright.

3

CSS frameworks

First how the browser picks a rule. Then what three authors say to do about it. Then one page, five ways.



GROUPS SELECT ONE
OF THE FOLLOWING
FRAMEWORKS

RESEARCH WHAT THE
VALUE PROPOSITION

GENERATE AN
EXAMPLE
PUT PPTX IN IN-CLASS

The words behind every framework

A rule's anatomy

Selector

which elements to style

Declaration

one property: value;

Declaration block

the declarations inside { }

Property / value

color: black; margin: 1rem;

Browser behaviour

Cascade

how competing rules are resolved

Specificity

the tie-breaker score

Inheritance

values flowing parent to child

Box model

content + padding + border + margin

Framework vocabulary

Component class

.card, .btn: a named pattern

Utility class

.mt-4, .text-center: one job

Breakpoint

a width where styles change

Design token

a named value: colour, spacing, type

```
.card { padding: 1rem; }      selector + declaration block -> property : value
```

A selector is a question asked of every element

“Do you match?” is asked of every element in the document, not just the one you had in mind.

THREE KINDS

```
h2          /* type */  
.badge      /* class */  
#header     /* id   */
```

COMBINATORS

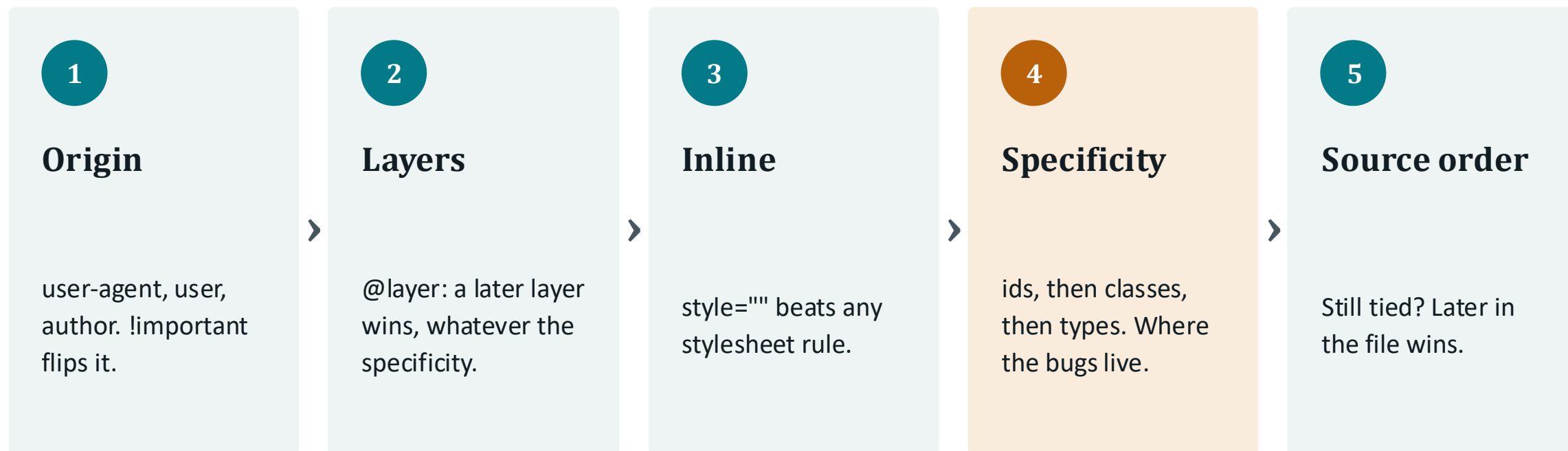
```
.nav a      /* descendant */  
.nav > a    /* child      */  
a.link      /* both, one  */
```

PSEUDO

```
a:hover  
li:first-child  
input:focus-visible
```

- When two rules match the same element and set the same property, someone has to lose. Deciding that is the cascade's job.

When two rules collide, these are checked in order



- The check stops at the first step that separates the two rules. A single id beats any number of classes before source order is even looked at.
- Everything in Grant's chapter 9 is a way of never reaching step 4 in anger.

Three counters, compared left to right

COUNT (ids, classes, types)

a	0,0,1	
.link	0,1,0	beats a
a.link	0,1,1	
.nav a	0,1,1	TIE -> source order
.nav .link	0,2,0	beats both
#main a	1,0,1	beats all of them

Pseudo-classes count as classes.

* and combinators count 0.

- Compared column by column, never summed: ten classes still lose to one id.
- `.nav a`` and `a.link`` tie, so whichever was written later wins. Move a file and the answer changes.
- Every fix that adds a class raises the score, and the next person has to raise it further. That is the escalation you felt in the drill.

If you cannot say why `.nav a`` beats `a.link``, reread ch. 1, section 1.1.3.

What flows down, and what wins at the end

Inheritance

Text properties flow to children: color, font, line-height. Box properties do not: margin, padding, border, background. An inherited value loses to any rule that matches the element directly.

Source order

Every earlier tie falls through to: later wins. Link states must be written :link, :visited, :hover, :active or hover never shows. Where you put the <link> tag is part of your CSS.

GRANT'S FOUR RULES OF THUMB (ch. 1)

1. No ids in selectors.
2. No !important.
3. Keep specificity low and flat.
4. Let source order settle the ties you meant.

Break one and the next person has to break it harder. !important is the end of the road.

Every rule is global. That is the whole problem.

Nothing in the language says “this rule is for the shelf label.” It is for every element that matches, forever, in files not written yet.

styles.css -- written for the shelf label

```
.badge {  
  border-radius: 1rem;  
  background: #d6f2dc; /* green */  
}  
.badge.expired { background: #fbd5d5; }
```

index.html -- the shopping list, a month later

```
<li>Rice    <span class="badge">2</span>  
<li>Coffee <span class="badge">1</span>  
  
-> both counts are GREEN.  
    Nobody asked for green.
```

- The fix `.shopping-list .badge` (0,2,0)` ties `.badge.expired`` and wins on source order, so Bread loses its red. You fixed what you were looking at and broke what you were not.
- Every framework that follows is a policy for never having this conversation.

Modular CSS: parts that never reach across

A module's styles do not depend on where it sits or what is around it. Change one module, nothing else moves.

1 Define the module

One class per module. Variations are modifiers: `.message`, `.message--error`. No type selectors inside; no rule reaching into another module.

2 Compose, don't nest

Bigger structures are modules inside modules, each owning its own layout. The module's name is in every class (`message__body`), so names are unique.

3 Scope, then document

`@scope` draws a boundary in the language itself. Then a pattern library, and a CSS-first workflow: build the parts, then the pages.

Class names are for developers, not for content

- Content semantics already live in the HTML elements. A class called `news` tells nobody anything new.
- So a class name cannot be “unsemantic.” It can only be semantic about something else: structure, role, reuse.
- The most reusable components have names independent of their content, and depend on neither DOM position nor element type.
- Multi-class: five button types × three sizes is nine classes composed on the element, not twenty-four.
- `js-\*` classes for behaviour, so a redesign cannot break a script.

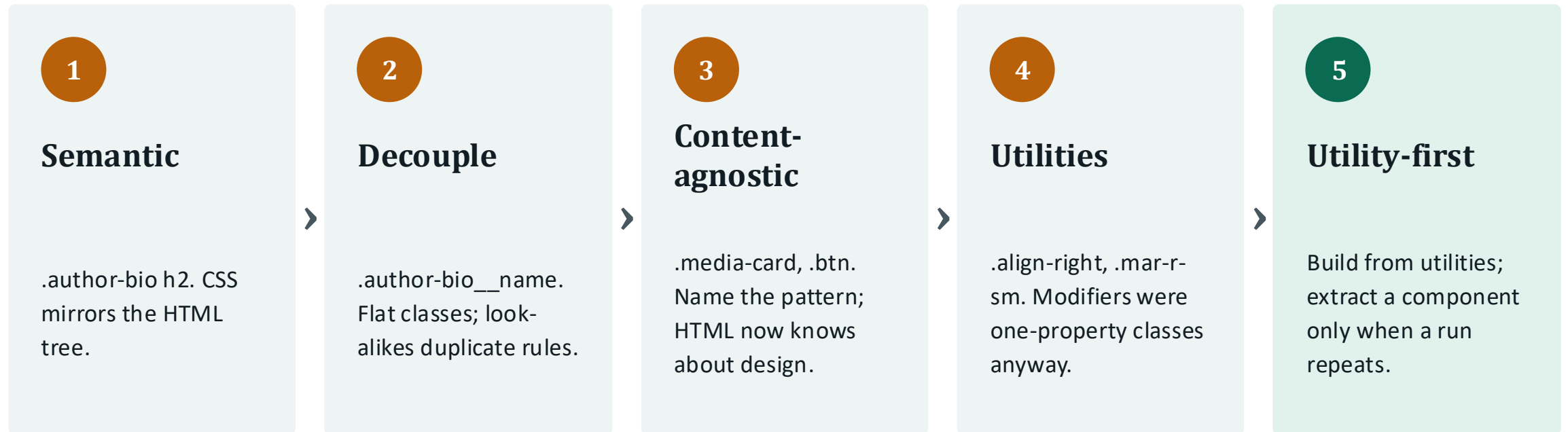
THE NAMING HE LANDED ON

```
.ComponentName  
.ComponentName--modifierName  
.ComponentName-descendant  
.is-stateName      /* toggled by JS */  
.u-utilityName     /* one job */
```

Key factor: reuse

More classes in the HTML, in exchange for CSS that does not grow with every page. Repetition compresses; alchemy does not.

He starts semantic and refactors his way off it



“Separation of concerns” was never the choice. The choice is which way the dependency points: CSS depends on HTML, or HTML depends on CSS.

Why utilities are not inline styles

Constraints beat freedom

Every new CSS file is a blank canvas.
Real projects: 124 to 402 distinct text colours. A utility set is a curated menu: text-sm exists, 13.5px does not.

CSS stops growing

When everyone picks from one scale, a new page adds HTML, not CSS. The stylesheet is bounded by the vocabulary, not the screen count.

Extract late

A component is earned by repetition, not designed up front. A navbar that appears once is not a component.

WHERE REUSE WENT

```
BEM / Bootstrap    reuse lives in the stylesheet  (.card defined once)
Tailwind           reuse lives in the template    (the foreach, the React component)
Neither removes reuse. They move it.
```

Each one is one of those essays, shipped as a file

	Plain CSS	BEM	Bootstrap	Tailwind
Class name says	whatever	what it is	which part	what it looks like
Who designed the look	you	you	them	you, from their scale
Where reuse lives	nowhere	the stylesheet	their stylesheet	your templates
Specificity fights	constant	none by rule	when you override	none
Cost you pay	global bugs	the naming tax	looks like Bootstrap	long strings; needs templates
Reading	Grant ch. 1	Gallagher · Grant ch. 9	Gallagher, productised	Wathan

Same pixels on screen in every column. The class names are the philosophy.

Questions the readings would ask your project

Who owns the design?

Bootstrap gives you parts; Tailwind gives you a scale and no parts.

Restyleable HTML or reusable CSS?

Content sites that get re-skinned lean semantic; apps with many screens lean utility.

Is there a template layer?

Utilities without templates are just long strings. PHP loops, Blade, React all count.

How do you escape?

Bootstrap: CSS variables first. Tailwind: extend the scale. Plain CSS: pray.

People and years?

Naming discipline survives turnover if enforced. A scale survives it without enforcement.

The phone?

Mobile-first is the default in both frameworks. In plain CSS it is a decision.

HW3: pick one, build a responsive page on it, say which question decided it.

Two things CSS added that the essays did not have

@layer (2022)

```
@layer framework, overrides;

@layer framework { .btn { padding: 1rem } }
@layer overrides { .btn { padding: 2rem } }

/* overrides wins: layer order is
   checked before specificity */
```

@scope (2024)

```
@scope (.card) to (.card__content) {
  img { border-radius: 1rem; }
}

/* img matches only inside .card
   and stops at .card__content */
```

- Neither replaces a framework. They remove the two reasons frameworks had to be clever: overriding a library without a specificity war, and keeping a module's rules inside the module.
- Tailwind 4 and Bootstrap 5.3 already put themselves in layers, so your one-class override beats theirs.

In-Class Exercise

In-class/wk4

One page, five ways

0

The problem

plain CSS: find the leak, fix it, break something else, escalate

1

BEM

names that say what a thing is: no collisions, a naming tax

2

Bootstrap

components someone else designed: finished parts, their look

3

Tailwind

names that say what it looks like: utilities, scales, reuse in the template

4

Responsive

the same page on a phone: mobile-first, prefixes, screenshots at three widths

- Same Docker image as week 3. ``docker compose watch`` for the pages, ``docker compose run --rm shots`` for the photographs.
- Hand in: ANSWERS.md, two questions, pushed before you leave.

The same badge, three ways

BEM -- what it is

```
<span class=
  "shelf-label
  shelf-label--expired">
expired</span>
```

BOOTSTRAP -- their part

```
<span class=
  "badge rounded-pill
  text-bg-danger">
expired</span>
```

TAILWIND -- what it looks like

```
<span class=
  "inline-block rounded-full
  px-2 py-0.5 text-xs
  bg-red-100 text-red-900">
expired</span>
```

- Same pixels on screen. Same HTML content. Only the class names differ, and the class names are a philosophy.
- Gallagher: a name should describe the thing. Wathan: that fiction never survives a real project. Grant: both are reacting to the same global problem.

Find the leak, fix it, break something else

Open `/0-plain/`. Look at the bottom of the page. Who asked for green counts?

THREE SELECTORS, THREE SCORES

```
.badge                0,1,0
.badge.expired        0,2,0
.shopping-list .badge  0,2,0  <- your fix. A TIE. Later wins. Bread loses its red.
.shopping-list .badge.expired 0,3,0  <- the next fix. Then a fourth rule. Then !important.
```

- Open DevTools on the Bread count: two rules, one struck through. That is the specificity lesson in one screenshot.
- Write ANSWERS.md question 1 now: what leaked, what the fix tied with, why Bread changed.

Each one fixes the previous one's pain

2009

OOCSS

separate structure
from skin; compose
classes

2010

BEM

block__element--
modifier: the
component's name
in every class

2011

Bootstrap

finished
components;
compose a page
from parts

2017

Tailwind

one declaration per
class; the look
written on the
element

2022

@layer

the language finally
gets an ordering
that beats
specificity

Grant ch. 9 walks this line. Today you run 2010, 2011 and 2017 on the same page.

Add a “low stock” label to Rice

1-bem -- every rule is 0,1,0

```
.shelf-label { ... }  
.shelf-label--expired { ... }  
.shopping-list__count { ... }  
no ties, ever. No descendants, ever.
```

THE COST IS IN index.html

```
<span class="shelf-label  
          shelf-label--expired">  
<span class="shopping-list__count  
          shopping-list__count--urgent">
```

- Amber, next to the shelf label. In Part 0 that was one rule; here you must name it first. shelf-label--low? A new block?
- Compare with a neighbour. Six people, six names for the same label. That is the finding.
- Reread the HTML: twice as long, every class invented and typed by a person.

Three changes, three costs

2-bootstrap -- no stylesheet of ours

```
<link href="../../../bootstrap.min.css">

<span class="badge rounded-pill text-bg-danger">
<button class="btn btn-outline-secondary">
```

HOW IT AVOIDED PART 0'S BUG

```
.badge      -> neutral base, no colour
.text-bg-*  -> colour is a separate modifier

their discipline, so you never count
```

- Softer red on “expired”: override `.text-bg-danger` and you are back in Part 0. Or set `--bs-danger-rgb` on `:root`, the door Bootstrap left open. Find the door.
- Denser table: `table-sm` exists. Check for the knob before writing CSS.
- Make it not look like Bootstrap: you cannot, not cheaply. That blue and that radius are its identity.

Names that say what it looks like

3-tailwind/index.html

```
<span class="inline-block rounded-full  
          px-2 py-0.5 text-xs  
          bg-red-100 text-red-900">  
  expired</span>
```

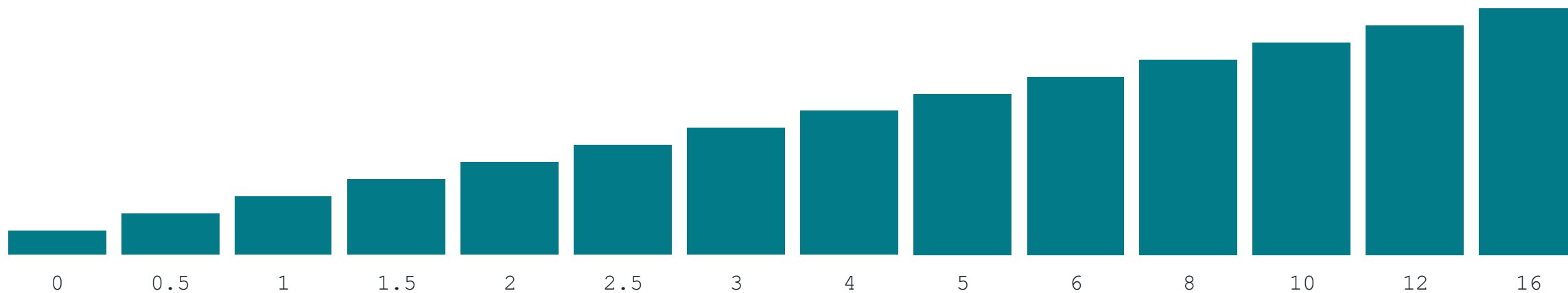
WHAT EACH ONE IS

rounded-full	border-radius: 9999px
px-2	padding-inline: 0.5rem
text-xs	font-size: 0.75rem
bg-red-100	a designed red

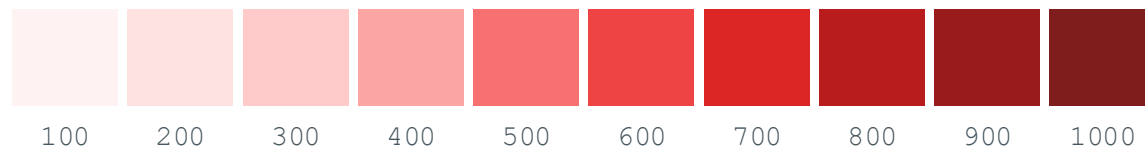
There is no .badge. There is no name for the thing at all. Nothing named, nothing to collide.

- Stop naming. Write the look on the element, one class per declaration. Wathan's phase 5, running.

You cannot type 13px, and that is a kindness



`p-{n}` `m-{n}` `gap-{n}` `w-{n}`: one scale for every spacing property, 0.25rem per step



red-100 to red-900: a designed ramp, not whatever hex was in the clipboard

- Two people on two pages end up matching, because neither could type a free value. That is a design system's whole job. These are design tokens; Bootstrap has them too (`--bs-*`).

Change the badge, then find the repetition

3-tailwind -- ten badges, seven classes each

```
<span class="inline-block rounded-full px-2 ...">  
<span class="inline-block rounded-full px-2 ...">  
... x10. Change the radius: ten edits.
```

week 3 index.php -- written ONCE

```
<?php foreach ($products as $p): ?>  
    <span class="badge <?= e($p['status']) ?>">  
<?php endforeach; ?>  
seven rows, one template.
```

- px-2 → px-3 → px-7. You are picking from a scale; notice which values do not exist.
- Change the radius on every badge. Count the edits. Then find where week 3's badge is written once.
- ANSWERS.md question 2: the BEM, Bootstrap and Tailwind badge. Choose one for HW3. Name what you give up.

375 pixels, no phone required

Pantry

Stock

Shopping list

Products

Tuesday, September 15. Seven products; two have gone off.

Not enough in stock.

Name

e.g. Milk

Best before

mm/dd/yyyy

Add product

Product	In stock	Best before		Buy	Use
Bread	1	2026-09-13	expired	1 Buy	1 Use
Butter	1	2026-09-17	due soon	1 Buy	1 Use
Cheese	0.5	2026-09-30	fresh	1 Buy	1 Use
Eggs	12	2026-09-19	due soon	1 Buy	1 Use
Milk	2.5	2026-09-25	fresh	1 Buy	1 Use
Rice	0	—	no date	1 Buy	1 Use
Yogurt	4	2026-09-08	expired	1 Buy	1 Use

Shopping list

Rice

2

Bread

1

Coffee

1

Pantry

Stock

Shopping list

Products

Tuesday, September 15. Seven products; two have gone off.

Not enough in stock.

Name

e.g. Milk

Best before

mm/dd/yyyy

Add product

Product	In stock		Buy
Bread	1	2026-09-13 expired	1 Buy
Butter	1	2026-09-17 due soon	1 Buy
Cheese	0.5	2026-09-30 fresh	1 Buy
Eggs	12	2026-09-19 due soon	1 Buy
Milk	2.5	2026-09-25 fresh	1 Buy
Rice	0	— no date	1 Buy
Yogurt	4	2026-09-08 expired	1 Buy

Shopping list

Rice

2

Bread

1

Coffee

1

- Left: designed for a desk, squeezed. Dates wrap, the button breaks, six columns fight over 375 px.
- Right: the same page. The form stacks; the date moved under the product name; the Use column waits for 640 px.
- Both from Chromium inside the container: `docker compose run --rm shots`. Fifteen PNGs, three widths.
- A screenshot is one expect() away from a test. That is Homework 3.

3-tailwind

4-responsive

No prefix is the phone; prefixes add width

TAILWIND

```
<form class="flex flex-col gap-3
      md:flex-row md:items-end">
<th class="hidden md:table-cell">
<span class="block md:hidden">2026-09-13
```

BOOTSTRAP -- same idea

```
<div class="col-12 col-md-6">
<th class="d-none d-md-table-cell">

sm 576  md 768  lg 992  xl 1200
```

- sm from 640, md 768, lg 1024, xl 1280. Breakpoints are a scale too: chosen once, so two pages break at the same widths.
- Hiding is not the only move. The date was not removed; it moved.
- Exercise: swap which action survives on the phone. Two prefixes. Re-shoot with the shots service.

SO WHICH ONE?

Three policies for one problem

	BEM	Bootstrap	Tailwind
Collisions	impossible by naming	avoided by their discipline	nothing named, nothing collides
What you write	names, rules, markup	their class names	the look, on the element
Your look or theirs	yours	theirs	yours
Where reuse lives	the CSS	their components	your templates
Hurts most when	every variant needs a name	the brand is yours	there is no template layer

HW3 lets you pick Bootstrap or Tailwind. Pick for the reason in the last row.

4

AI and design work

An assistant produces CSS fast and cannot see the page. You can see the page. The loop that works is built on that.

Know which job you are handing over

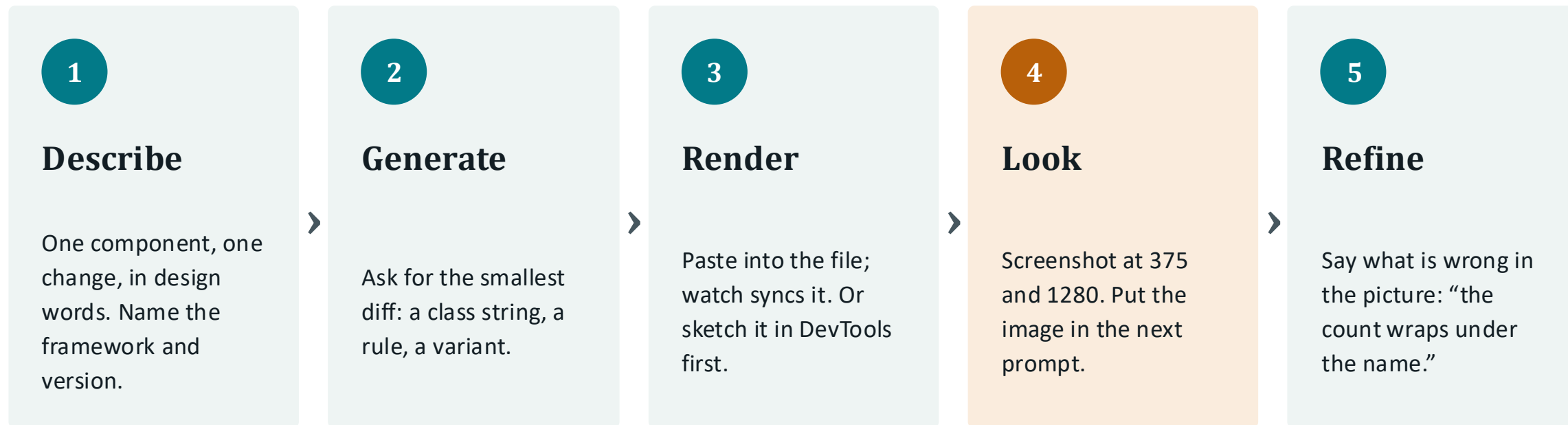
Hand it these

- A sentence into utilities: “a pill badge, small text, soft red” → the class string.
- Explaining a cascade conflict from a pasted Styles pane.
- Translating: this BEM block as Bootstrap; this Bootstrap card as Tailwind.
- Variants: three spacings, two colour treatments, so you can look instead of type.
- Mobile-first prefixes for a layout you describe in words.

Keep these

- Seeing. It does not know what rendered unless you show it a screenshot.
- Your constraints: scale, tokens, framework version. Tell it every time.
- Your existing CSS. It will write a fourth rule rather than fix the first.
- Taste and the brief. “Make it look better” returns something.
- The escalation reflex: a longer selector, then !important.

Describe, generate, render, look, refine



↪ back to 1, with the screenshot

Step 4 is the one people skip. An iteration without a render is a guess stacked on a guess.

Five parts, every time

ON THE PART 3 BADGE

CONTEXT Tailwind 4 via the browser CDN. Mobile-first.
 <span class="inline-block rounded-full px-2
 py-0.5 text-xs bg-red-100 text-red-900">

GOAL Read as a warning, not an error: amber, same
 size and shape.

BOUNDARY Change only the class string. No <style>,
 no arbitrary values like [13px]. Default scale.

OUTPUT The new class string, then one line on why.

CHECK Here is the 375px screenshot. The badge wraps
 under the name. Fix that.

- Context is what it cannot see. Half of bad output is a missing version number.
- Goal in design words gets a better answer than “change bg-red-100.”
- Boundary is where your constraints live. Without it: a <style> block and a magic number.
- Check is the loop: the screenshot goes in, the next diff comes out.

Three things to leave with

1

CSS is global

Every rule matches everything that fits. Specificity decides ties; source order decides the rest. If you are counting, you are already losing.

2

Frameworks are policies

BEM: name everything. Bootstrap: use our parts. Tailwind: name nothing, put reuse in the template. All three answer Part 0.

3

Phone first, screenshot always

Unprefixed is the phone. A headless browser at 375 px is a screenshot away, and a screenshot is a test away. Same rule for AI output: see it before you believe it.